



Conservatoire National des Arts et Métiers

Master 2 Actuariat

Scoring du risque de sinistre en assurance voyage

Machine Learning, approche lift et agent IA d'aide à la
souscription

Auteurs : Hamza Assal

Abdou Ndiaye

Projet : Assurance voyage et agent IA d'aide à la souscription

Application : Prototype Shiny et agent IA explicatif

Date : 2026

Table des matières

1	Fondements théoriques et contexte actuariel	4
1.1	Problématique métier et objectif actuariel	4
1.2	Modéliser un sinistre rare : enjeux statistiques	5
1.3	Modèles supervisés et logique probabiliste	5
1.4	XGBoost et modèles de gradient boosting	5
1.5	SMOTE et apprentissage sur classes déséquilibrées	6
1.6	Validation croisée, échantillon test et bootstrap	7
1.7	Métriques adaptées aux sinistres rares et au scoring	7
2	Modélisation du risque de sinistre	8
2.1	Données et préparation du portefeuille	8
2.1.1	Nettoyage des données	8
2.1.2	Analyse exploratoire	8
2.1.3	Déséquilibre de classes	9
2.1.4	Préprocessing	9
2.2	Modèle complet avec variables tarifaires	9
2.2.1	Gestion du déséquilibre par SMOTE	9
2.2.2	Optimisation des hyperparamètres	12
2.2.3	Choix du modèle champion complet	13
2.2.4	Comparaison des modèles	14
2.2.5	Résultats du modèle champion complet	14
2.2.6	Analyse de seuils du modèle complet	15
2.2.7	Validation bootstrap du modèle complet	15
2.3	Modèle applicatif sans variables tarifaires	16
2.3.1	Motivation méthodologique	16
2.3.2	Résultats de la deuxième approche	17
2.3.3	Pourquoi SMOTE n'améliore pas le modèle sans variables tarifaires ?	17
2.3.4	Pouvoir de scoring du modèle sans variables tarifaires	18
3	Agent IA explicatif et application Shiny	20
3.1	Pourquoi ajouter un agent IA au modèle de scoring ?	20
3.2	Qu'est-ce qu'un agent IA dans ce projet ?	20
3.3	Construction de l'agent IA	21

3.4	Architecture applicative Shiny	21
3.5	Valeur ajoutée pour l'underwriting	22
3.6	Limites et gouvernance de l'agent	22
3.7	Discussion actuarielle et limites	23
3.7.1	Limites du modèle	23
3.8	Conclusion	23

Bibliographie méthodologique indicative	24
--	-----------

Liste des Figures

Liste des Tables

Mots-clés : assurance voyage ; sinistre rare ; scoring du risque ; aide à la souscription ; Machine Learning ; XGBoost ; SMOTE ; bootstrap ; validation croisée ; déséquilibre de classes ; ROC AUC ; PR AUC ; lift ; SHAP ; agent IA ; application Shiny ; underwriting.

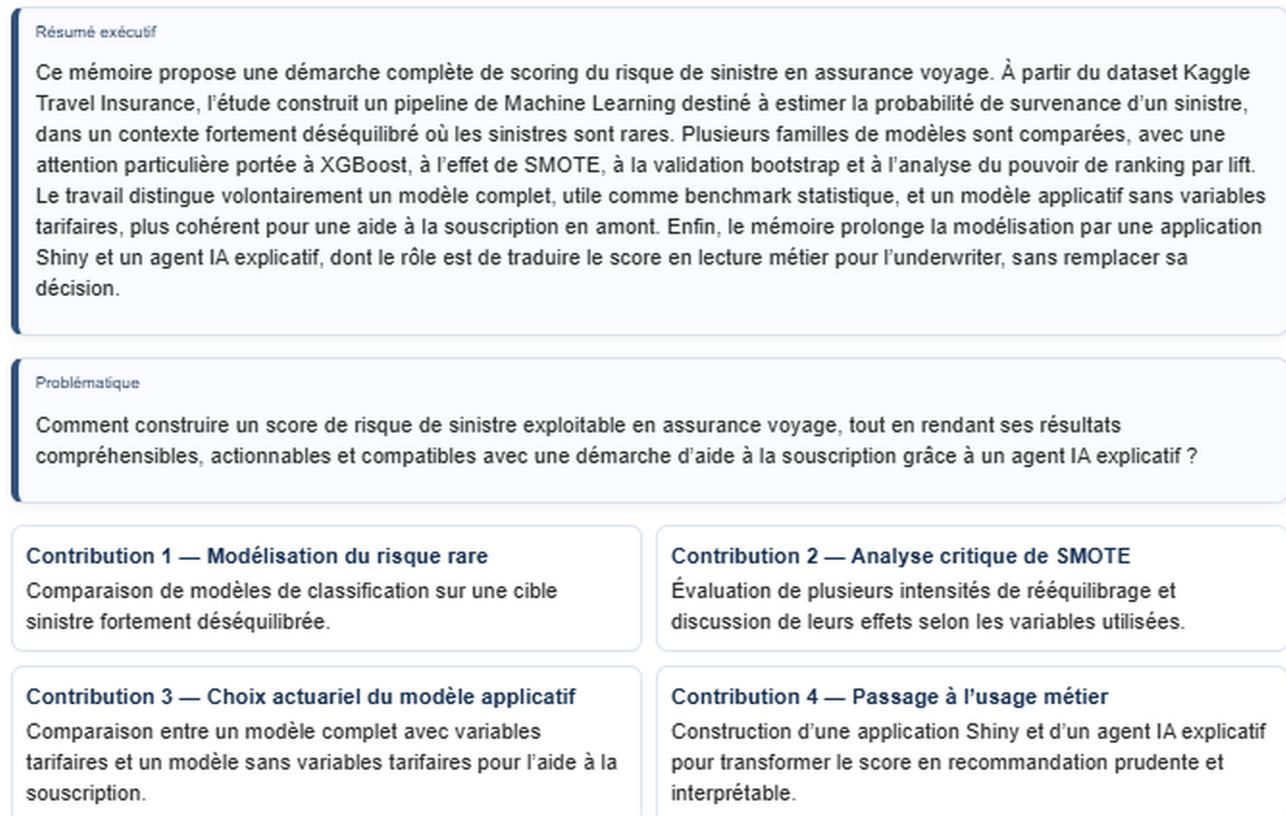


FIGURE 1 – Résumé exécutif, problématique et contributions du mémoire.

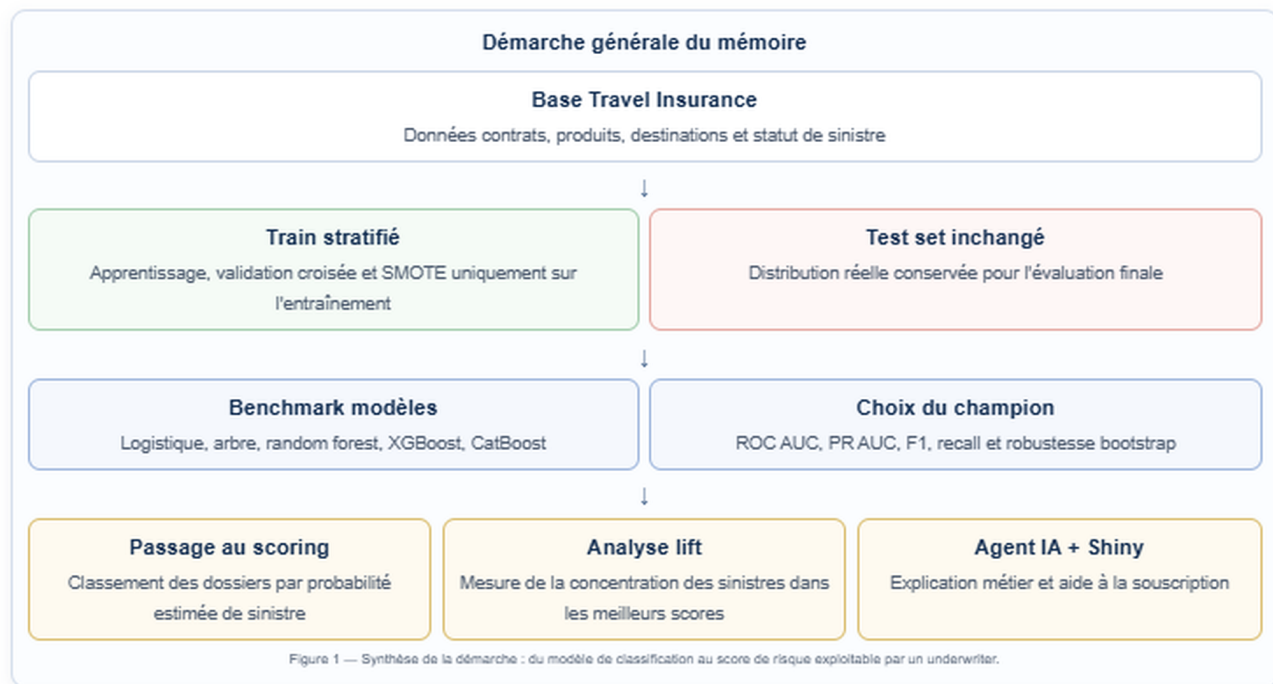


FIGURE 2 – Démarche générale du mémoire : de la base initiale au scoring et à l'agent IA.

1 Fondements théoriques et contexte actuariel

Ce mémoire étudie l'application du Machine Learning à l'assurance voyage, avec un objectif opérationnel : estimer la probabilité de survenance d'un sinistre. Le projet s'inscrit dans une démarche actuarielle appliquée, où le modèle n'est pas uniquement un outil prédictif, mais aussi un support d'analyse du risque et d'aide à la décision.

La cible du projet est `claim_status`, construite à partir de la variable de statut de sinistre disponible dans la base. La classe positive est `yes`, qui correspond à la présence d'un sinistre, et la classe négative est `no`.

1.1 Problématique métier et objectif actuariel

En assurance voyage, les sinistres sont généralement rares mais financièrement et opérationnellement importants. Pouvoir attribuer une probabilité de sinistre à un contrat permet d'alimenter des analyses de segmentation, de surveillance du portefeuille, de tarification exploratoire et de priorisation des contrôles.

Le modèle ne remplace pas le jugement actuariel. Il fournit un score de risque qui doit être interprété avec les limites de la base, la politique de souscription, les garanties couvertes et les contraintes réglementaires.

1.2 Modéliser un sinistre rare : enjeux statistiques

La prédiction de sinistres relève d'un problème de classification binaire très déséquilibré. La majorité des observations correspond à une absence de sinistre, tandis que la classe **yes** représente une fraction très faible du portefeuille. Cette structure pose trois difficultés principales.

Premièrement, le modèle peut apprendre à privilégier la classe majoritaire. Dans ce cas, il obtient une accuracy élevée en prédisant presque toujours **no**, mais il détecte mal les dossiers sinistrés. Deuxièmement, les métriques usuelles deviennent moins lisibles : une bonne accuracy ne signifie pas nécessairement une bonne capacité de détection du risque. Troisièmement, la rareté des sinistres rend l'estimation plus instable, car quelques observations positives peuvent influencer fortement les mesures de recall, de précision ou de F1-score.

Pour cette raison, la démarche retenue combine plusieurs outils : validation croisée stratifiée, métriques adaptées aux classes rares, analyse de seuils, analyse de lift, bootstrap et comparaison entre stratégies de rééquilibrage. Le modèle est évalué comme un outil de scoring et non uniquement comme un classifieur binaire au seuil de 50 %.

1.3 Modèles supervisés et logique probabiliste

Le cadre général est celui de l'apprentissage supervisé. On observe, pour chaque dossier i , un vecteur de caractéristiques X_i et une variable cible Y_i indiquant la présence ou l'absence de sinistre. Le modèle cherche à approximer la probabilité conditionnelle :

$$P(Y = yes \mid X = x).$$

Cette probabilité estimée peut ensuite être utilisée de deux façons. La première consiste à fixer un seuil de décision et à transformer le score en classe prédite. La seconde, plus adaptée à ce projet, consiste à utiliser directement le score pour hiérarchiser les dossiers. En assurance, cette deuxième lecture est souvent plus pertinente : l'enjeu n'est pas seulement de prédire **yes** ou **no**, mais d'identifier les dossiers qui méritent une analyse prioritaire.

La modélisation ne se réduit donc pas à la maximisation d'une métrique unique. Elle doit arbitrer entre détection des sinistres, maîtrise des faux positifs, robustesse statistique et lisibilité métier. C'est cette logique qui justifie la comparaison entre modèles classiques, modèles d'ensemble, rééquilibrage par SMOTE, tuning raisonnable et analyse de lift.

1.4 XGBoost et modèles de gradient boosting

XGBoost appartient à la famille des méthodes de **gradient boosting**. Le principe général consiste à construire un modèle prédictif fort en agrégeant progressivement plusieurs modèles faibles, généralement des arbres de décision. Chaque nouvel arbre est entraîné pour corriger les erreurs résiduelles des arbres précédents. Le modèle final est donc une somme d'arbres, construite de manière séquentielle.

D'un point de vue intuitif, un arbre de décision découpe l'espace des variables en zones homogènes. Le boosting répète cette opération plusieurs fois : les premiers arbres capturent les effets les plus structurants, puis les arbres suivants se concentrent sur les erreurs restantes. XGBoost optimise une

fonction de perte et ajoute des termes de régularisation afin de contrôler la complexité du modèle. Cette régularisation est essentielle en assurance, car le modèle peut facilement surapprendre des segments rares ou des combinaisons spécifiques présentes dans l'échantillon.

Les principaux hyperparamètres mobilisés dans ce mémoire ont chacun un rôle précis. Le nombre d'arbres (**trees**) contrôle la taille de l'ensemble. La profondeur des arbres (**tree_depth**) détermine le niveau d'interaction que le modèle peut apprendre. Le taux d'apprentissage (**learn_rate**) ralentit ou accélère la contribution de chaque arbre. Les paramètres de taille minimale de noeud, de sous-échantillonnage des lignes et de sous-échantillonnage des variables limitent le surapprentissage. Enfin, la pondération de classe peut être utilisée pour donner davantage d'importance aux sinistres dans la fonction objectif.

L'intérêt de XGBoost tient à sa capacité à capturer naturellement des relations non linéaires et des interactions entre variables. En assurance voyage, l'effet d'une destination, d'une durée de voyage, d'un produit ou d'un canal de distribution peut dépendre d'autres caractéristiques du dossier. Une relation strictement linéaire serait souvent trop restrictive. Dans ce projet, XGBoost est donc utilisé non seulement comme classifieur, mais surtout comme **fonction de scoring** : il attribue à chaque dossier une probabilité estimée de sinistre, ensuite utilisée pour hiérarchiser le portefeuille.

1.5 SMOTE et apprentissage sur classes déséquilibrées

Le problème étudié est fortement déséquilibré : les sinistres représentent une part très faible des observations. Dans ce contexte, un modèle peut obtenir une accuracy élevée tout en détectant très peu de sinistres. La classe minoritaire **yes** doit donc faire l'objet d'une attention particulière.

SMOTE, pour *Synthetic Minority Over-sampling Technique*, est une méthode de rééquilibrage qui crée des observations synthétiques de la classe minoritaire. Contrairement à une simple duplication, SMOTE génère de nouveaux points par interpolation entre observations minoritaires proches. L'idée est de densifier localement l'espace des sinistres afin d'aider le modèle à mieux identifier les zones associées à un risque plus élevé.

Dans le pipeline, SMOTE est appliqué uniquement sur les données d'entraînement, ou à l'intérieur des folds de validation croisée. Il n'est jamais appliqué au test set, afin de conserver une évaluation réaliste sur la distribution originale du portefeuille. Cette distinction est essentielle pour éviter une fuite de données : le test set doit représenter ce que le modèle rencontrerait en situation réelle, avec la rareté naturelle des sinistres.

SMOTE n'est toutefois pas une solution automatiquement meilleure. Un rééquilibrage trop fort peut déformer la distribution d'apprentissage et conduire le modèle à produire trop d'alertes. Il peut aussi être moins pertinent lorsque certaines variables sont catégorielles encodées, car l'interpolation dans un espace transformé ne correspond pas toujours à une observation métier naturelle. Dans ce mémoire, SMOTE améliore le benchmark complet lorsque les variables tarifaires sont conservées, mais il n'améliore pas l'approche sans variables tarifaires. Cette différence est méthodologiquement importante : elle montre que le rééquilibrage doit être testé empiriquement, et non appliqué de manière systématique.

1.6 Validation croisée, échantillon test et bootstrap

La validation croisée permet d'estimer la performance d'un modèle de façon plus stable qu'une seule séparation apprentissage/test. Dans ce projet, une validation croisée stratifiée en cinq folds est utilisée : le train est découpé en plusieurs sous-échantillons, en conservant autant que possible la proportion de sinistres dans chaque fold. À chaque itération, le modèle est entraîné sur quatre folds et évalué sur le fold restant. Les métriques sont ensuite moyennées.

Cette approche n'est pas en conflit avec l'utilisation d'un test set final. Les deux niveaux répondent à deux objectifs différents. La validation croisée sert à comparer les modèles, calibrer certains seuils et choisir une configuration. Le test set, lui, reste isolé jusqu'à l'évaluation finale et sert à estimer la performance du modèle retenu sur des observations non utilisées pour l'apprentissage ni pour le choix du modèle.

Cette séparation est particulièrement importante dans un mémoire actuariel, car elle limite le risque d'optimisme dans l'évaluation. Si l'on choisissait le modèle directement sur le test set, celui-ci deviendrait implicitement un outil de sélection et ne fournirait plus une mesure indépendante de généralisation. La validation croisée joue donc le rôle de laboratoire de comparaison, tandis que le test set joue le rôle d'épreuve finale.

Le bootstrap complète cette logique. Il consiste à tirer plusieurs échantillons avec remise à partir du train, à réentraîner le modèle champion, puis à mesurer la variabilité des performances sur le test set inchangé. L'objectif du bootstrap n'est pas de corriger le déséquilibre de classes. Il sert plutôt à observer comment les métriques varient lorsque l'échantillon d'entraînement change légèrement. Si les performances sont très instables d'une réplcation à l'autre, le modèle doit être interprété avec prudence. Si elles restent relativement concentrées, cela renforce la confiance dans la robustesse du score.

1.7 Métriques adaptées aux sinistres rares et au scoring

Dans un problème de sinistres rares, l'accuracy seule est insuffisante. Un modèle qui prédit systématiquement l'absence de sinistre peut obtenir une très bonne accuracy, mais il est inutilisable pour détecter les dossiers risqués. Le mémoire mobilise donc plusieurs métriques complémentaires : recall, précision, F1-score, ROC AUC, PR AUC, analyse de seuils et lift.

Le recall mesure la part des sinistres effectivement détectés parmi tous les sinistres observés. Il est important lorsqu'un faux négatif représente un dossier risqué non identifié. La précision mesure la part de vrais sinistres parmi les dossiers signalés comme risqués ; elle est importante pour éviter de saturer l'underwriter avec trop de faux positifs. Le F1-score synthétise précision et recall, mais il reste dépendant du seuil de classification retenu.

La ROC AUC mesure la capacité générale du modèle à ordonner les sinistres avant les non-sinistres, indépendamment d'un seuil unique. La PR AUC est souvent plus informative lorsque la classe positive est rare, car elle se concentre sur la performance autour de la classe minoritaire. Le lift a une interprétation directement métier : il indique combien un segment de dossiers scorés comme risqués est plus sinistré que la moyenne du portefeuille.

L'approche finale du projet est donc orientée **scoring/ranking** plutôt que décision automatique. Un bon modèle actuariel dans ce contexte n'est pas nécessairement celui qui maximise l'accuracy, mais celui qui concentre utilement le risque dans les premiers segments de score. Cette logique est

cohérente avec un usage d'aide à la décision : prioriser les dossiers, expliciter le niveau de risque et laisser la décision finale à l'underwriter.

2 Modélisation du risque de sinistre

2.1 Données et préparation du portefeuille

TABLE 1 – Dimensions de la base nettoyée

n_rows	n_cols
55284	11

Les détails de structure et de complétude des variables sont disponibles en les fichiers de sortie du dossier **reports/**.

La base contient des informations relatives à l'agence, au canal de distribution, au produit, à la durée, à la destination, au montant des ventes nettes, à la commission, au genre et à l'âge. Les noms de colonnes sont standardisés avec `janitor::clean_names()`.

2.1.1 Nettoyage des données

Le script d'import lit le CSV disponible dans **data/raw/**, standardise les colonnes, détecte la cible, supprime les doublons exacts et sauvegarde **data/processed/data_clean.rds**. Le fichier local contient une colonne **Claim**, renommée en **claim_status** afin de conserver une nomenclature stable dans tout le pipeline.

Les valeurs manquantes détaillées sont reportées en les fichiers de sortie du dossier **reports/**.

2.1.2 Analyse exploratoire

Voir Figure A.1 - Distribution de la cible.

TABLE 2 – Distribution de claim_status

claim_status	n	share
no	54363	0.9833
yes	921	0.0167

Les graphiques détaillés des variables numériques sont conservés dans le dossier **reports/figures/** afin de ne pas alourdir le corps du mémoire.

Les graphiques détaillés des variables catégorielles sont conservés dans le dossier **reports/figures/** afin de ne pas alourdir le corps du mémoire.

La matrice de corrélation complète reste disponible dans le dossier **reports/figures/**.

2.1.3 Déséquilibre de classes

La proportion de sinistres observés est de **** 1.67 % **** contre **** 98.33 % **** de contrats sans sinistre. Cette rareté rend l'accuracy insuffisante : un modèle qui prédit toujours **no** obtient une accuracy élevée, mais ne détecte aucun sinistre.

Dans ce contexte, le recall sur la classe **yes** devient une métrique métier importante : il mesure la part des sinistres effectivement détectés. La précision reste également utile, car elle indique la proportion de prédictions positives réellement sinistrées. La ROC AUC est retenue pour classer les modèles sur leur capacité globale de discrimination.

2.1.4 Préprocessing

La séparation train/test est réalisée avant tout apprentissage du preprocessing, avec une stratification sur `claim_status`. Le test set reste non modifié pendant tout le projet.

La recette `tidymodels` applique l'imputation, le regroupement des modalités rares, la gestion des nouvelles modalités, l'encodage one-hot, la suppression des variables à variance nulle et la normalisation des variables numériques. Cette organisation évite les fuites de données, car les paramètres de preprocessing sont appris uniquement sur les données d'entraînement ou sur les folds de validation.

2.2 Modèle complet avec variables tarifaires

Le benchmark compare deux stratégies :

- **none** : apprentissage sans rééquilibrage ;
- **smote** : application de SMOTE uniquement à l'intérieur des folds de validation croisée et sur le train lors du fit final.

Les modèles effectivement entraînés sont : régression logistique, arbre de décision, random forest, XGBoost et CatBoost. CatBoost a nécessité une installation depuis la release officielle du projet, car le package n'était pas disponible via CRAN pour R 4.6.0 dans l'environnement local. Les résultats CatBoost sont donc produits dans un script dédié et comparés au champion XGBoost.

Une précaution méthodologique doit être signalée : `net_sales` et `commission` peuvent porter une information redondante, la commission étant généralement liée au montant de prime ou de vente nette. Le modèle complet est donc interprété comme un benchmark prédictif utilisant toute l'information disponible, et non comme le modèle le plus propre pour une aide à la souscription en amont.

La validation croisée est stratifiée en 5 folds. Les métriques calculées sont : accuracy, precision, recall, F1-score, ROC AUC et PR AUC. Les métriques sensibles à la classe événement sont calculées explicitement sur la classe positive **yes**.

2.2.1 Gestion du déséquilibre par SMOTE

SMOTE est utilisé comme stratégie de comparaison pour augmenter artificiellement la représentation de la classe rare dans les données d'entraînement. Il n'est jamais appliqué au test set.

L'upsampling et le downsampling ne sont pas retenus dans le pipeline principal afin de garder une comparaison plus resserrée : **none** donne le point de référence, tandis que **smote** teste une approche synthétique adaptée aux classes rares. Le downsampling aurait supprimé de l'information dans une base déjà peu riche en sinistres, et l'upsampling simple aurait dupliqué des observations sans créer de diversité locale.

Effet de SMOTE sur le déséquilibre de classes

Dans le pipeline, SMOTE est appelé explicitement dans `scripts/04_model_training.R` avec `themis::step_smote(claim_status, over_ratio = 0.30, skip = TRUE)`. Il est ajouté à une recette dédiée, `recipe_smote`, après la séparation train/test réalisée dans `scripts/03_preprocessing.R`.

Ainsi, SMOTE intervient uniquement sur les données d'entraînement : soit lors du fit final sur le train set, soit à l'intérieur des folds de validation croisée gérés par `tidymodels`. Le test set n'est jamais rééquilibré et conserve sa distribution réelle. Cette organisation évite une fuite de données, car aucune observation synthétique construite à partir de l'information d'entraînement ne vient contaminer l'évaluation finale.

SMOTE ne duplique pas simplement les lignes minoritaires existantes. Il crée des observations synthétiques de la classe **yes** par interpolation locale entre observations minoritaires proches, ce qui enrichit l'espace d'apprentissage tout en limitant la simple répétition d'exemples.

Le tableau détaillé de l'effet de SMOTE sur la distribution des classes est disponible en [Tableau B.1](#).

Voir [Figure A.2](#) - Effet de SMOTE sur la balance des classes.

Vérification de l'utilisation effective de SMOTE

Un audit complémentaire vérifie que les modèles associés à la stratégie **smote** utilisent bien une base d'entraînement rééquilibrée. Pour la stratégie **none**, les modèles sont entraînés sur le train original non rééquilibré. Pour la stratégie **smote**, la recette contient explicitement `themis::step_smote(claim_status, over_ratio = 0.30, skip = TRUE)`, puis elle est préparée sur l'échantillon d'entraînement.

Le résultat observé confirme que les configurations **smote** utilisent 56 539 observations d'entraînement, dont 13 047 sinistres synthétiques ou originaux, contre 44 227 observations et 735 sinistres pour la stratégie **none**. Le test set reste identique pour toutes les configurations, avec 11 057 observations et 186 sinistres.

L'audit détaillé des données d'entraînement utilisées par stratégie est disponible en [Tableau B.2](#).

Le rapport retient ici la version de benchmark dans laquelle les configurations sont classées par validation croisée stratifiée selon la ROC AUC, puis le F1-score et le recall sur la classe **yes**. Dans cette version, `xgboost__smote` est le modèle retenu : il présente la meilleure ROC AUC du classement et un recall nettement supérieur à la version sans rééquilibrage.

Les tableaux d'audit recalculés configuration par configuration sur le test set final restent disponibles dans les fichiers CSV du projet, mais ils ne sont pas affichés dans cette section afin de conserver une lecture cohérente du scénario retenu pour le mémoire : le benchmark complet avec variables tarifaires et stratégie SMOTE de référence.

TABLE 3 – Benchmark retenu pour le rapport : classement de validation croisée

config_id	model	strategy	roc_auc	f_meas	recall
xgboost__smote	xgboost	smote	0.8090	0.1429	0.4435
xgboost__none	xgboost	none	0.8073	0.1446	0.2054
logistic_regression__none	logistic_regression	none	0.8011	0.1501	0.2626
logistic_regression__smote	logistic_regression	smote	0.7959	0.1375	0.4857
random_forest__smote	random_forest	smote	0.7923	0.1384	0.4544
random_forest__none	random_forest	none	0.7909	0.1404	0.2259
decision_tree__smote	decision_tree	smote	0.7602	0.1396	0.4762
decision_tree__none	decision_tree	none	0.4892	0.0327	1.0000

Sensibilité à l'intensité du rééquilibrage SMOTE

L'analyse précédente utilise un SMOTE avec `over_ratio` = 0.30, ce qui revient à viser un ratio minoritaire / majoritaire d'environ 30 %. En proportion totale, cela produit environ 23 % de sinistres dans l'échantillon d'entraînement préparé. Cette intensité est nettement supérieure à la fréquence réelle des sinistres, proche de 1,7 %, et peut donc déformer la distribution d'apprentissage.

SMOTE n'est pas une méthode automatiquement meilleure. Il peut aider le modèle à voir davantage de cas minoritaires, mais un rééquilibrage trop fort peut augmenter les faux positifs, dégrader la précision, perturber le ranking des scores ou réduire la PR AUC. Pour cette raison, une analyse complémentaire teste deux intensités plus modérées :

- `smote_10` : `over_ratio` = 0.111, soit environ 10 % de sinistres et 90 % de non-sinistres ;
- `smote_20` : `over_ratio` = 0.25, soit environ 20 % de sinistres et 80 % de non-sinistres.

Le test set reste inchangé pour toutes les configurations. Le choix final ne doit donc pas se limiter à une seule métrique : il doit tenir compte à la fois des métriques de classification, de la PR AUC et de l'analyse de ranking/lift.

Cette sous-section ne correspond pas au benchmark principal `none` versus `smote` avec `over_ratio` = 0.30. Elle teste volontairement des intensités plus modérées de SMOTE, nommées `smote_10` et `smote_20`, afin de documenter la sensibilité de la méthode au niveau de rééquilibrage. Ces variantes sont présentées comme des contrôles méthodologiques ; le scénario utilisé pour la sélection principale reste le benchmark de référence, dans lequel le modèle retenu est `xgboost__smote`.

Les proportions obtenues selon les intensités de SMOTE sont disponibles en [Tableau B.3](#).

Les tests `smote_10` et `smote_20` sont conservés comme analyse de sensibilité méthodologique : ils vérifient qu'un SMOTE plus modéré ne produit pas mécaniquement un meilleur modèle. Pour éviter de mélanger plusieurs scénarios de sélection, les tableaux détaillés de ces variantes ne sont pas repris dans le corps du rapport. Le résultat utilisé pour la comparaison principale reste le benchmark de référence où `xgboost__smote` est retenu.

Les résultats du benchmark complet montrent que SMOTE peut apporter un gain lorsque toutes les variables disponibles sont conservées. Dans cette configuration, le meilleur modèle est `xgboost__smote` : il présente une ROC AUC légèrement supérieure à `xgboost__none` et un recall nettement plus élevé sur la classe `yes`. Cette stratégie est donc retenue comme champion statistique du modèle complet.

Cette lecture doit toutefois rester critique. SMOTE n'est pas une méthode automatiquement meilleure : son efficacité dépend de la structure des variables utilisées par le modèle. Dans le modèle complet, les variables tarifaires comme `net_sales` et `commision_in_value` structurent fortement l'espace des observations. Elles fournissent des signaux continus et fortement informatifs, ce qui peut rendre les observations synthétiques créées par SMOTE plus cohérentes pour XGBoost. Dans ce cas, SMOTE aide le modèle à mieux apprendre une frontière de risque autour de la classe rare.

En revanche, lorsque les variables tarifaires sont retirées, le modèle s'appuie davantage sur des variables catégorielles encodées en one-hot et sur des signaux moins directement discriminants. SMOTE peut alors interpoler dans un espace moins naturel : certaines observations synthétiques deviennent moins représentatives de vrais profils métier. Cette hypothèse explique pourquoi SMOTE améliore le modèle complet mais ne gagne pas dans l'approche sans variables tarifaires. Le résultat est important pour le mémoire : il montre que SMOTE doit être considéré comme une hypothèse de modélisation à tester, et non comme un correctif automatique au déséquilibre de classes.

2.2.2 Optimisation des hyperparamètres

Une étape complémentaire d'optimisation des hyperparamètres a été ajoutée afin de vérifier si le modèle champion pouvait être amélioré de manière raisonnable. L'objectif n'est pas de réaliser une recherche exhaustive, mais de produire une comparaison propre, reproductible et présentable dans un mémoire actuariel.

Le tuning est limité aux familles de modèles les plus pertinentes pour ce projet : XGBoost sans SMOTE, XGBoost avec pondération de classe, XGBoost avec SMOTE léger, Random Forest et CatBoost. CatBoost est inclus dans les tableaux de comparaison au même titre que les autres familles de modèles. Son entraînement est réalisé via l'API native R, car cette API ne s'intègre pas directement au workflow `tidymodels` utilisé pour XGBoost et Random Forest.

La validation croisée reste stratifiée en 5 folds pour XGBoost et Random Forest. Pour CatBoost, une première tentative plus large en 5-fold s'est avérée trop coûteuse localement ; une grille contrôlée a donc été exécutée en validation croisée stratifiée 3-fold. Les métriques prioritaires sont la ROC AUC et la PR AUC, car le problème est fortement déséquilibré. Les métriques de classification au seuil, comme le F1-score, le recall et la précision, sont également calculées, mais elles ne suffisent pas à elles seules pour juger un modèle orienté scoring. Le lift dans le Top 10 % des scores reste une métrique métier importante, car l'application vise à prioriser les dossiers les plus risqués.

Pour XGBoost, les hyperparamètres testés sont : `trees`, `tree_depth`, `learn_rate`, `loss_reduction`, `min_n`, `sample_size` et `mtry`. Une variante avec pondération de classe est aussi testée via `scale_pos_weight`, avec plusieurs valeurs dont la valeur empirique `no / yes` observée sur le train. Cette pondération est une alternative à SMOTE : elle conserve les observations originales mais modifie le poids relatif des erreurs sur la classe minoritaire.

Pour Random Forest, la grille porte sur `mtry`, `trees` et `min_n`. Le tuning reste volontairement limité afin d'éviter une optimisation trop lourde et peu lisible dans le cadre du mémoire.

Pour CatBoost, les stratégies testées sont `none`, `smote` et `weighted`. Les versions `none` et `weighted` exploitent directement les variables catégorielles via l'API native CatBoost. La version `smote` utilise la recette de preprocessing avec `step_smote()` uniquement sur le train, ce qui impose une

représentation encodée des variables. La pondération de classe teste plusieurs valeurs du poids de la classe **yes**, dont le ratio empirique **no / yes**.

Les tableaux complets du tuning restent sauvegardés dans les fichiers CSV dédiés (**reports/hyperparameter_tuning**, **reports/best_tuned_models.csv**, **reports/tuned_model_test_metrics.csv**, **reports/tuned_model_lift** et **reports/champion_vs_tuned_comparison.csv**). Pour garder une lecture cohérente du rapport principal, cette section ne réaffiche pas les variantes historiques ou intermédiaires qui ne correspondent pas au scénario final retenu. La conclusion utilisée dans le mémoire est la suivante : aucun modèle tuné ne remplace le champion du benchmark complet, **xgboost__smote**.

Les résultats montrent que le tuning ne justifie pas de remplacer le modèle champion actuel. Le meilleur candidat tuné est un XGBoost pondéré, mais il n'améliore pas le champion sur le test set : la ROC AUC et la PR AUC restent légèrement inférieures, et le lift Top 10 % est comparable mais non supérieur. Surtout, le seuil retenu par validation croisée pour ce modèle est très conservateur : il améliore la précision mais détecte très peu de sinistres, ce qui réduit fortement son intérêt opérationnel.

Le modèle XGBoost avec SMOTE reste le meilleur benchmark prédictif global lorsque les variables tarifaires sont conservées. Pour l'application Shiny, la version sans variables tarifaires est toutefois privilégiée afin d'éviter une recommandation fondée sur une prime ou une commission déjà calculée. Dans les tableaux comparatifs, CatBoost fait partie des candidats évalués au même niveau que les autres modèles. Il apporte des arbitrages intéressants, notamment sur le recall dans les versions pondérées ou rééquilibrées, mais il ne dépasse pas le champion complet **xgboost__smote** sur le critère de sélection statistique.

2.2.3 Choix du modèle champion complet

Cette section consolide les dernières évolutions du projet : sensibilité à l'intensité de SMOTE, ajout effectif de CatBoost et optimisation raisonnable des hyperparamètres. Le choix final ne repose pas uniquement sur une métrique de classification binaire. Dans ce contexte de sinistres rares, la ROC AUC, la PR AUC et les indicateurs de ranking, en particulier le lift dans le Top 10 % et le Top 20 %, sont essentiels pour juger l'utilité opérationnelle du modèle.

TABLE 4 – Synthèse finale de sélection du modèle

Candidat	Strategie	ROC AUC	PR AUC	Recall	Lift 10 %
xgboost	smote	0.8218	0.0845	0.414	4.8374

Lecture du tableau. Le modèle retenu dans le benchmark complet est la version XGBoost avec SMOTE. Il est privilégié pour sa capacité à mieux rappeler les sinistres tout en conservant une discrimination satisfaisante.

La synthèse finale distingue le champion prédictif et le champion applicatif. Le benchmark complet retient **xgboost__smote** : c'est le meilleur candidat statistique lorsque l'on conserve toute l'information disponible, y compris les variables tarifaires. L'analyse complémentaire sans variables tarifaires retient en revanche XGBoost sans SMOTE : dans cette seconde approche, SMOTE ne parvient pas à dépasser la stratégie **none**. Cette différence est cohérente avec la nature des variables utilisées : SMOTE semble plus efficace dans l'espace complet, structuré par des variables

continues très informatives, que dans l'espace sans tarif, plus fortement dominé par des variables catégorielles encodées.

2.2.4 Comparaison des modèles

Le classement complet des configurations est disponible en [Tableau B.4](#).

Le tableau de classement ci-dessus est la synthèse utilisée pour la décision : il agrège les résultats du benchmark selon la règle de sélection retenue. Les métriques détaillées par configuration restent sauvegardées dans `reports/model_comparison.csv`, mais elles ne sont pas affichées ici pour éviter de multiplier les lectures partielles du même benchmark.

Le choix du modèle champion suit l'ordre défini : ROC AUC, puis F1-score, puis recall sur **yes**. Cette règle privilégie d'abord la capacité de discrimination des probabilités, puis la qualité de classification de la classe sinistre.

Un audit de la logique ML a conduit à calibrer un seuil de décision par configuration à partir des prédictions out-of-fold. Cette étape est importante : comparer les F1-scores au seuil arbitraire de 0,5 aurait pénalisé les modèles capables de discriminer les sinistres mais conservateurs dans leurs probabilités.

Les seuils calibrés par configuration sont disponibles en [Tableau B.5](#).

2.2.5 Résultats du modèle champion complet

Le modèle champion est `**xgboost__smote`. **Le seuil de décision utilisé pour les métriques finales est 0.48****, calibré sur les prédictions out-of-fold de validation croisée afin de maximiser le F1-score sans utiliser le test set.

TABLE 5 – Métriques finales sur le test set non modifié

Metrique	Valeur
accuracy	0.9153
precision	0.0852
recall	0.4140
f_meas	0.1413
roc_auc	0.8218
pr_auc	0.0845

TABLE 6 – Matrice de confusion sur le test set

Prediction	Truth	n
no	no	10044
yes	no	827
no	yes	109
yes	yes	77

Prediction	Truth	n
------------	-------	---

Voir Figure A.3 - Matrice de confusion.

Voir Figure A.4 - Courbe ROC.

Voir Figure A.5 - Courbe précision/rappel.

2.2.6 Analyse de seuils du modèle complet

Une analyse complémentaire a été réalisée sur le test set pour comprendre l'effet du seuil de décision du modèle champion XGBoost. Les seuils testés vont de 0,01 à 0,30 par pas de 0,01.

Cette analyse illustre un arbitrage central en assurance : diminuer le seuil augmente le nombre de sinistres détectés, donc le recall, mais augmente aussi les faux positifs. À l'inverse, augmenter le seuil améliore généralement la précision, mais laisse davantage de sinistres non détectés.

TABLE 7 – Seuils sélectionnés : statistique et métier

Type	Seuil	Accuracy	Precision	Recall	F1	Positifs prédits	FP	FN
statistical_f1_max	0.10	0.9664	0.1477	0.2097	0.1733	264	225	147
business_recall_min_500	0.05	0.9082	0.0924	0.5054	0.1563	1017	923	92

Le seuil statistique maximise le F1-score. Il cherche un équilibre entre precision et recall. Le seuil métier impose une contrainte différente : atteindre au moins 50 % de recall lorsque cela est possible. Ce second seuil accepte davantage de faux positifs afin de capter une part plus importante des sinistres.

Voir Figure A.6 - F1-score selon le seuil.

Les courbes complémentaires de précision/rappel par seuil restent disponibles dans `reports/figures/`.

Les courbes détaillées de faux positifs et faux négatifs par seuil restent disponibles dans `reports/figures/`.

La courbe du nombre de positifs prédits par seuil reste disponible dans `reports/figures/`.

Le tableau complet de l'analyse de seuils est disponible en le fichier `reports/threshold_analysis.csv`.

2.2.7 Validation bootstrap du modèle complet

Le bootstrap est appliqué uniquement au modèle champion pour mesurer la stabilité statistique de ses performances. Il ne sert pas à corriger le déséquilibre de classes. À chaque réplication, un échantillon bootstrap du train est tiré, le modèle champion est réentraîné, puis évalué sur le test set initial non modifié.

TABLE 8 – Résumé bootstrap du modèle champion

Metrique	Moyenne	Ecart-type	Q2.5 %	Mediane	Q97.5 %
accuracy	0.9496	0.0055	0.9397	0.9502	0.9591
f_meas	0.1595	0.0102	0.1386	0.1594	0.1801
pr_auc	0.0903	0.0056	0.0795	0.0903	0.0995
precision	0.1116	0.0092	0.0971	0.1106	0.1299
recall	0.2838	0.0331	0.2258	0.2849	0.3415
roc_auc	0.8233	0.0046	0.8148	0.8234	0.8318

Voir [Figure A.7 - Distribution bootstrap de la ROC AUC](#).

Les distributions bootstrap du F1-score et du recall restent disponibles dans `reports/figures/`.

Les distributions bootstrap complémentaires restent disponibles dans `reports/figures/`.

2.3 Modèle applicatif sans variables tarifaires

2.3.1 Motivation méthodologique

L'analyse SHAP du modèle complet, reportée en [Figure A.9](#), fait apparaître une question méthodologique importante : les variables tarifaires, notamment `net_sales` et `commision_in_value`, sont très informatives pour prédire la survenance d'un sinistre. Cette information est utile d'un point de vue prédictif, mais elle doit être interprétée avec prudence dans une application destinée à un underwriter.

En effet, la prime nette et la commission peuvent déjà refléter une partie du processus commercial ou tarifaire. Si le modèle utilise ces variables pour produire une recommandation, il existe un risque de circularité : le dossier est jugé risqué en partie parce qu'il a déjà été vendu, tarifié ou commissionné selon une logique qui intègre elle-même des éléments de risque. Cette situation n'est pas forcément problématique pour un benchmark prédictif, mais elle est moins satisfaisante pour un outil d'aide à la décision qui doit expliquer le risque à partir des caractéristiques du dossier.

Pour cette raison, une deuxième approche a été construite dans `scripts/no_tariff/01_no_tariff_benchmark`. Elle retire les variables tarifaires disponibles avant de reconstruire les recettes de preprocessing et de relancer le benchmark. Dans les données actuellement préparées, les variables effectivement retirées sont `net_sales` et `commision_in_value`. Le script vérifie aussi les variantes `commision` et `commission` si une autre version du dataset les utilise.

Cette deuxième approche répond donc à une question différente du benchmark initial :

- le modèle complet cherche la meilleure performance prédictive avec toute l'information disponible ;
- le modèle sans variables tarifaires cherche une lecture plus prudente pour l'underwriting, en évitant de fonder la recommandation sur une prime ou une commission déjà calculée.

Le classement détaillé des modèles sans variables tarifaires est disponible en [Tableau B.6](#).

TABLE 9 – Comparaison des champions : modèle complet vs modèle sans variables tarifaires

Approche	Modele	ROC AUC	Recall	Lift 10 %
Complet	xgboost smote	0.8218	0.4140	4.8374
Sans tarif	xgboost none	0.8179	0.1559	5.0524

Lecture du tableau. Le modèle complet conserve le meilleur candidat statistique avec SMOTE. Le modèle sans variables tarifaires est retenu pour l'application Shiny, car il évite de fonder l'aide à la souscription sur une prime ou une commission déjà calculée.

2.3.2 Résultats de la deuxième approche

Les résultats montrent que le champion de l'approche sans variables tarifaires est XGBoost sans SMOTE. Le retrait de `net_sales` et `commision_in_value` entraîne une baisse de performance prédictive par rapport au modèle complet, ce qui confirme que ces variables concentraient une information importante. Le modèle sans variables tarifaires conserve toutefois une capacité de scoring exploitable et présente une lecture plus prudente pour l'application Shiny.

La comparaison des deux approches est donc la suivante : dans le modèle complet, le champion retenu est XGBoost avec SMOTE ; dans le modèle sans variables tarifaires, le champion est XGBoost sans SMOTE. Ce résultat ne constitue pas une contradiction, mais une information méthodologique importante : l'intérêt de SMOTE dépend fortement de l'espace de variables dans lequel il est appliqué.

2.3.3 Pourquoi SMOTE n'améliore pas le modèle sans variables tarifaires ?

SMOTE crée des observations synthétiques de la classe minoritaire en interpolant entre des observations existantes. Cette logique fonctionne mieux lorsque l'espace d'apprentissage est suffisamment continu et structuré. Dans le modèle complet, les variables tarifaires comme `net_sales` et `commision_in_value` fournissent justement des signaux numériques continus, fortement liés à la sinistralité. Elles peuvent aider XGBoost à construire une frontière de risque plus stable autour des sinistres synthétiques générés par SMOTE.

Lorsque ces variables sont retirées, l'information disponible repose davantage sur des variables catégorielles encodées en one-hot, comme l'agence, le canal de distribution, le type de produit ou la destination. Dans un tel espace, l'interpolation synthétique devient moins naturelle : les profils créés par SMOTE peuvent correspondre à des combinaisons de modalités moins réalistes ou moins discriminantes métier. Le modèle peut alors apprendre du bruit ou des frontières artificielles qui dégradent le ranking des scores.

Une autre explication est liée au signal prédictif disponible. Sans les variables tarifaires, le modèle dispose de moins d'information continue pour séparer les sinistres des non-sinistres. Ajouter des observations synthétiques ne compense pas nécessairement cette perte d'information ; au contraire, cela peut augmenter les faux positifs ou affaiblir la calibration des probabilités. Dans cette configuration, le déséquilibre de classes reste un problème, mais SMOTE n'est pas le correctif le plus efficace.

Ce point est important d'un point de vue actuariel : SMOTE ne doit pas être présenté comme une solution automatique au déséquilibre de classes. Il s'agit d'une hypothèse de modélisation à tester empiriquement. Ici, il améliore le benchmark complet avec variables tarifaires, mais il ne s'impose pas dans l'approche sans variables tarifaires. Cette différence renforce la nécessité de comparer plusieurs architectures de variables avant de choisir le modèle à déployer.

Pour un usage purement prédictif, le modèle complet avec SMOTE reste le meilleur candidat statistique dans la version retenue du benchmark. Pour une application underwriter-ready, la version sans variables tarifaires reste cependant plus prudente, car elle réduit le risque de circularité entre tarification déjà calculée et conseil donné à l'underwriter.

Voir Figure A.8 - Comparaison des champions complet et sans variables tarifaires.

2.3.4 Pouvoir de scoring du modèle sans variables tarifaires

Pourquoi analyser le modèle comme un score ?

Le modèle sans variables tarifaires n'est pas retenu parce qu'il produit une classification binaire parfaite. Le problème reste fortement déséquilibré : les sinistres sont rares dans le test set, avec un taux moyen proche de 1,68 %. Dans ce contexte, la classification directe au seuil donne une vision utile mais partielle. L'enjeu métier est plutôt de savoir si le modèle permet de hiérarchiser les dossiers et de concentrer les sinistres dans les segments de score les plus élevés.

Cette analyse est particulièrement pertinente pour l'application Shiny, car l'outil applicatif utilise le modèle sans variables tarifaires. Le score doit donc être présenté comme un signal de priorisation du risque, et non comme une décision automatique.

Principe de l'analyse par score

Le modèle XGBoost sans variables tarifaires produit une probabilité estimée de sinistre pour chaque observation du test set. Les dossiers sont ensuite triés par score décroissant. Les segments Top 1 %, Top 5 %, Top 10 %, Top 20 % et Top 30 % correspondent aux dossiers ayant les scores de risque les plus élevés. On mesure ensuite combien de sinistres réels sont contenus dans ces segments.

Les indicateurs utilisés sont les suivants :

- **Observations** : nombre de dossiers dans le segment considéré.
- **Sinistres capturés** : nombre de vrais sinistres présents dans le segment.
- **% des sinistres capturés** : part des sinistres totaux du test set retrouvés dans le segment.
- **Taux de sinistre segment** : nombre de sinistres du segment divisé par le nombre d'observations du segment.
- **Taux moyen test** : fréquence globale des sinistres dans le test set.
- **Lift** : rapport entre le taux de sinistre du segment et le taux moyen de sinistre du test set.

Formellement : $\text{Lift} = \text{taux de sinistre du segment} / \text{taux moyen de sinistre du test set}$.

TABLE 10 – Concentration des sinistres par score - modèle sans variables tarifaires

Segment	Obs.	Sinistres	% capturés	Taux segment	Taux test	Lift
Top 1 %	111	15	8.06 %	13.51 %	1.68 %	8.03
Top 5 %	553	54	29.03 %	9.76 %	1.68 %	5.80
Top 10 %	1106	94	50.54 %	8.5 %	1.68 %	5.05
Top 20 %	2212	132	70.97 %	5.97 %	1.68 %	3.55
Top 30 %	3318	145	77.96 %	4.37 %	1.68 %	2.60

Lecture détaillée des résultats

Pour le modèle XGBoost sans variables tarifaires, les 1 % de dossiers les plus risqués contiennent 15 sinistres sur 111 observations, soit 8,06 % des sinistres du test set. Le taux de sinistre dans ce segment atteint 13,51 %, contre 1,68 % en moyenne, ce qui correspond à un lift de 8,03. Ce segment constitue donc un groupe de très forte vigilance, même sans utiliser la prime ou la commission.

Le Top 5 % contient 54 sinistres sur 553 observations, soit 29,03 % des sinistres du test set. Son taux de sinistre est de 9,76 %, avec un lift de 5,80. Autrement dit, si un underwriter ne peut analyser qu'une faible fraction du portefeuille, le score permet déjà de concentrer près d'un tiers des sinistres observés dans 5 % des dossiers.

Le Top 10 % contient 94 sinistres sur 1 106 observations, soit 50,54 % des sinistres. Le taux de sinistre y atteint 8,50 %, avec un lift de 5,05. Ce résultat est central pour l'usage métier : même sans variables tarifaires, le modèle parvient à concentrer plus de la moitié des sinistres dans les 10 % de dossiers les plus risqués.

Le Top 20 % contient 132 sinistres, soit 70,97 % des sinistres du test set. Le taux de sinistre du segment est de 5,97 %, avec un lift de 3,55. Ce segment constitue un compromis opérationnel intéressant si l'assureur dispose d'une capacité d'analyse plus large.

Enfin, le Top 30 % contient 147 sinistres, soit 79,03 % des sinistres observés. Le taux de sinistre reste de 4,43 %, avec un lift de 2,63. Le gain marginal diminue, mais le segment demeure nettement plus risqué que la moyenne du portefeuille.

Interprétation métier

Cette analyse montre que le modèle sans variables tarifaires est plus pertinent comme outil de scoring que comme règle de décision binaire stricte. Sa valeur métier réside dans sa capacité à ordonner les dossiers par niveau de risque. Pour un underwriter, cela permet de prioriser les dossiers à relire, d'organiser les contrôles, de cibler les analyses complémentaires et de justifier une vigilance renforcée sur certains profils.

Le point important est que cette priorisation est obtenue sans utiliser la prime nette ni la commission. Le score repose donc davantage sur les caractéristiques intrinsèques du dossier : agence, canal, produit, destination, durée du voyage, âge et autres variables disponibles avant tarification. Cette propriété rend le modèle plus cohérent pour une application d'aide à la souscription.

Le modèle ne doit toutefois pas être interprété comme une certitude individuelle de sinistre. Le score signale une concentration statistique du risque dans certains segments ; il ne remplace ni les

règles internes de souscription, ni l'expertise humaine de l'underwriter. En définitive, l'analyse de lift du modèle sans variables tarifaires justifie son usage dans l'application Shiny : il permet de prioriser les dossiers tout en évitant une recommandation circulaire fondée sur une prime ou une commission déjà calculée.

3 Agent IA explicatif et application Shiny

3.1 Pourquoi ajouter un agent IA au modèle de scoring ?

Le modèle statistique produit une probabilité de sinistre et un score de priorisation. Cette sortie est utile pour classer les dossiers, mais elle reste technique : un underwriter doit comprendre ce que signifie le score, pourquoi le dossier mérite une attention particulière, quelles limites entourent la prédiction et quelle action métier raisonnable peut être envisagée. C'est précisément le rôle de la couche agent IA : transformer une sortie de modèle en explication opérationnelle, structurée et prudente.

Dans ce mémoire, l'agent IA ne remplace pas le modèle de machine learning et ne modifie pas la probabilité prédite. Il intervient après le calcul du score. Il reçoit le contexte du dossier, le résultat du modèle, le segment de risque, le lift, les principales contributions explicatives et les limites connues. Il reformule ensuite ces éléments dans un langage métier adapté à un chargé d'affaires, un souscripteur ou un actuaire.

L'intérêt de cette couche applicative est double. D'une part, elle rend le modèle plus utilisable par un public métier non spécialiste du machine learning. D'autre part, elle limite le risque de mauvaise interprétation en rappelant systématiquement que le score est une aide à l'analyse et non une décision automatique.

3.2 Qu'est-ce qu'un agent IA dans ce projet ?

Un agent IA peut être défini comme un système logiciel capable de recevoir un contexte, d'appliquer des instructions, de raisonner sur une tâche donnée et de produire une réponse adaptée à un objectif. Dans une version avancée, un agent peut utiliser des outils, consulter des bases documentaires, appeler des fonctions ou conserver un historique d'interaction. Dans le cadre de ce projet, l'agent est volontairement conçu comme un agent explicatif et prudent : il ne prend pas d'initiative décisionnelle, mais il interprète le score et accompagne l'utilisateur.

Concrètement, l'agent combine trois éléments :

- un **prompt système**, qui définit son rôle, son ton, ses limites et la structure attendue de la réponse ;
- un **contexte métier**, construit automatiquement à partir du dossier saisi et de la prédiction du modèle ;
- un **mécanisme de génération de réponse**, soit par appel à un LLM si une clé API est disponible, soit par un mode fallback local déterministe.

Cette architecture permet de séparer clairement la prédiction statistique et l'explication métier. Le modèle XGBoost calcule un score ; l'agent explique ce score. Cette séparation est importante en assurance : elle évite de donner à l'agent un rôle décisionnel qu'il ne doit pas avoir.

3.3 Construction de l'agent IA

La construction de l'agent repose sur plusieurs fichiers du projet. Le fichier `agent/agent_prompt.md` contient les instructions générales : l'agent doit se comporter comme un assistant spécialisé en underwriting assurance voyage, expliquer la probabilité prédite, interpréter le segment de risque, comparer le dossier au taux moyen du portefeuille, expliquer le lift et formuler une recommandation prudente. Le prompt interdit explicitement de conclure qu'un sinistre est certain ou de prendre une décision automatique.

La fonction `build_agent_context()` construit ensuite le contexte envoyé à l'agent. Ce contexte contient les caractéristiques du dossier, la probabilité estimée de sinistre, le seuil utilisé, la classe prédite, le segment de risque, le taux de sinistre historique du segment, le lift, la recommandation déterministe et les limites du modèle. Le fait de construire ce contexte de manière explicite réduit le risque de réponse vague ou déconnectée de la prédiction réelle.

La fonction `explain_prediction()` produit une explication locale simple, même sans appel externe à un LLM. Elle reformule le score, le compare au taux moyen du portefeuille et rappelle que le résultat est probabiliste. Lorsque des contributions SHAP sont disponibles, elles peuvent être intégrées pour indiquer les familles de variables qui influencent le score. Ces contributions restent descriptives : elles expliquent le comportement du modèle mais ne démontrent pas une causalité actuarielle.

La fonction `generate_recommendation()` fournit une recommandation underwriting déterministe selon le niveau de risque : revue prioritaire, analyse complémentaire, revue standard ou absence d'alerte particulière. Cette fonction joue un rôle de garde-fou : même si l'appel LLM échoue, l'application continue à fournir une réponse cohérente et maîtrisée.

Enfin, `call_llm_agent()` orchestre l'appel à l'agent. Si une clé API est disponible, la fonction peut interroger un LLM en lui transmettant le prompt système et le contexte du dossier. Si aucune clé n'est disponible, ou si l'appel échoue, le mode fallback local est activé. Cette robustesse est essentielle pour une démonstration académique : l'application reste fonctionnelle même sans dépendre d'un service externe.

3.4 Architecture applicative Shiny

L'application Shiny constitue l'interface métier du projet. Elle permet à l'utilisateur de saisir les caractéristiques d'un dossier, de calculer le score de risque, d'afficher le segment associé, puis d'interroger l'agent IA. L'application utilise volontairement le modèle applicatif sans variables tarifaires, chargé depuis `models/no_tariff_best_model.rds` avec ses métadonnées `models/no_tariff_model_metadata.rds`.

Ce choix distingue le champion prédictif du champion applicatif. Le modèle complet avec variables tarifaires reste le meilleur benchmark statistique dans la version retenue. En revanche, pour une application d'aide à l'underwriting, le modèle sans variables tarifaires est plus cohérent : il évite de fonder la recommandation sur une prime ou une commission déjà calculée. L'interface ne demande donc plus la prime ni la commission dans le formulaire de saisie.

La logique applicative peut être résumée ainsi :

Ce schéma résume la séparation des responsabilités. Le modèle de Machine Learning produit un score ; l'application organise ce score dans une interface métier ; l'agent IA reformule et contex-

tualise l'information. L'agent n'a donc pas vocation à modifier la prédiction, mais à rendre son interprétation plus accessible et plus directement exploitable par un souscripteur.

La page de scoring affiche la probabilité estimée de sinistre, le seuil opérationnel, le segment de risque, le taux de sinistre historique du segment, le lift et une recommandation synthétique. L'onglet agent IA permet ensuite à l'utilisateur de poser une question en langage naturel, par exemple : pourquoi ce dossier est-il risqué, que signifie le lift, ou quelles sont les limites du modèle ?

3.5 Valeur ajoutée pour l'underwriting

L'apport principal de l'agent IA est la traduction opérationnelle du score. Un modèle peut produire une probabilité de 8 %, mais cette valeur n'a de sens pour un underwriter que si elle est comparée au taux moyen du portefeuille, replacée dans un segment de risque et accompagnée d'une recommandation prudente. L'agent peut ainsi expliquer qu'un dossier appartient à un segment plus sinistré que la moyenne, que le lift indique une concentration du risque, et qu'une revue complémentaire peut être justifiée.

L'agent contribue aussi à la traçabilité de l'analyse. En structurant sa réponse en synthèse du risque, éléments explicatifs, recommandation underwriting, points de vigilance et limites, il rend l'utilisation du modèle plus contrôlable. Cette structure est importante pour un usage assurantiel, où les décisions doivent rester justifiables et compatibles avec les règles internes de souscription.

Enfin, l'agent facilite la pédagogie autour du modèle. Il peut rappeler que le score ne prédit pas avec certitude un sinistre, qu'il existe des faux positifs et des faux négatifs, et que le modèle doit être surveillé dans le temps en cas de changement de portefeuille. Il transforme donc une sortie algorithmique en support d'aide à la décision.

3.6 Limites et gouvernance de l'agent

La principale limite est que l'agent dépend du contexte qui lui est transmis. Il ne doit pas inventer de variables, de règles contractuelles ou de décisions juridiques. Il doit rester strictement aligné sur les données du dossier et sur les résultats du modèle. Pour cette raison, le prompt système impose plusieurs garde-fous : ne jamais affirmer qu'un sinistre est certain, ne jamais remplacer l'underwriter, ne jamais formuler une décision automatique définitive.

Une autre limite concerne l'explicabilité. Les contributions SHAP améliorent la transparence du score, mais elles ne constituent pas une preuve causale. Une variable peut contribuer fortement à la prédiction sans être une cause directe du sinistre. L'agent doit donc présenter ces éléments comme des signaux du modèle et non comme des vérités actuarielles définitives.

Dans une mise en production réelle, l'agent devrait être encadré par une gouvernance spécifique : validation du prompt, journalisation des réponses, contrôle des dérives, audit des recommandations, suivi des performances du modèle et validation par les équipes métier. Le prototype présenté dans ce mémoire constitue donc une preuve de concept : il montre comment articuler machine learning, scoring du risque et explication assistée par IA, sans transférer la décision finale à l'algorithme.

3.7 Discussion actuarielle et limites

Les résultats confirment la difficulté d'un problème de sinistres rares. La ROC AUC du modèle champion indique une capacité de discrimination exploitable, mais la précision et le F1-score restent modestes, ce qui est cohérent avec une faible prévalence de la classe **yes**.

D'un point de vue métier, le score peut être utilisé comme indicateur de vigilance plutôt que comme décision automatique. Un contrat avec une probabilité élevée de sinistre pourrait justifier une analyse complémentaire, une surveillance renforcée ou une étude des facteurs de risque associés. Le recall est central si l'objectif est de limiter les sinistres non détectés, tandis que la précision devient prioritaire si l'on veut réduire les faux positifs dans une procédure opérationnelle coûteuse.

Dans une perspective de mise en production, une étape complémentaire importante serait l'étude de la calibration probabiliste. Un modèle peut très bien classer les dossiers du plus risqué au moins risqué, tout en produisant des probabilités imparfaitement calibrées. Pour un usage actuariel, cette distinction est essentielle : le ranking sert à prioriser, tandis que la calibration sert à interpréter le niveau absolu du risque. Une analyse future pourrait donc comparer les probabilités prédites aux taux de sinistre observés par déciles de score, puis recalibrer le modèle si nécessaire.

La gouvernance du dispositif constitue également un point central. Le score devrait être suivi dans le temps afin de détecter une éventuelle dérive du portefeuille, une modification des comportements de sinistres ou un changement dans la distribution des produits vendus. L'agent IA devrait, de son côté, être encadré par des règles de réponse, une journalisation des échanges et une validation métier régulière. Cette supervision garantit que l'outil reste une aide à la souscription et non une boîte noire décisionnelle.

3.7.1 Limites du modèle

La principale limite est le déséquilibre extrême de la cible. Les métriques de classification dépendent fortement du seuil de décision. Un seuil unique ne reflète pas nécessairement les coûts asymétriques entre faux positifs et faux négatifs.

CatBoost a finalement pu être installé depuis une release officielle et exécuté localement. La principale limite technique restante est le coût de calcul : une grille CatBoost large en validation croisée 5-fold s'est révélée trop longue dans l'environnement disponible. Le rapport conserve donc une comparaison CatBoost sérieuse mais volontairement limitée, en 3-fold stratifié, afin de ne pas bloquer la reproductibilité du projet.

3.8 Conclusion

Le projet met en place un pipeline reproductible pour prédire la survenance d'un sinistre en assurance voyage. Les scripts produisent une base nettoyée, une analyse exploratoire, un benchmark de modèles, une évaluation finale sur test set, une sauvegarde du modèle champion et une validation bootstrap.

Les résultats montrent que la discrimination probabiliste est plus informative que l'accuracy brute dans ce contexte. Le modèle champion constitue une base exploitable pour un mémoire actuariel et pour une future application Shiny enrichie par un agent IA explicatif.

Bibliographie méthodologique indicative

Cette bibliographie regroupe des références classiques et utiles pour justifier les choix méthodologiques du mémoire. Elle couvre l'apprentissage statistique, les modèles utilisés, la gestion du déséquilibre de classes, l'interprétabilité et quelques références orientées assurance.

Apprentissage statistique et machine learning général

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning : Data Mining, Inference, and Prediction*. Springer. Référence centrale pour les fondements de l'apprentissage statistique, la régularisation, les arbres, les méthodes d'ensemble et l'évaluation des modèles.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning*. Springer. Ouvrage pédagogique utile pour présenter la régression logistique, les arbres, les forêts aléatoires, la validation croisée et les métriques de performance.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer. Référence appliquée pour la construction de pipelines prédictifs, le preprocessing, le tuning et la comparaison de modèles.
- Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly. Référence pratique pour l'écosystème `tidymodels`, utilisé dans le projet pour structurer recettes, workflows, validation croisée et métriques.

Modèles utilisés dans le benchmark

- Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5-32. Article fondateur des forêts aléatoires, utilisées dans le benchmark comme modèle non linéaire robuste.
- Friedman, J. H. (2001). Greedy Function Approximation : A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232. Article fondateur du gradient boosting, base conceptuelle des méthodes modernes de boosting.
- Chen, T., & Guestrin, C. (2016). XGBoost : A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Référence principale pour XGBoost, modèle central du projet.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost : unbiased boosting with categorical features. *NeurIPS*. Référence principale pour CatBoost, testé dans le benchmark et le tuning.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273-297. Référence fondatrice des SVM, famille de modèles souvent utilisée dans les benchmarks classiques de classification.

Données déséquilibrées, SMOTE et évaluation

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE : Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357. Article fondateur de SMOTE, méthode utilisée pour traiter la rareté des sinistres dans le benchmark.

- He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284. Synthèse importante sur les problèmes de classification déséquilibrée.
- Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *Proceedings of ICML*. Référence utile pour justifier l'utilisation conjointe de ROC AUC et PR AUC dans les problèmes rares.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874. Référence classique sur l'interprétation des courbes ROC.

Interprétabilité et SHAP

- Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*. Article fondateur de SHAP, utilisé pour interpréter les contributions des variables au score.
- Molnar, C. (2022). *Interpretable Machine Learning*. Ouvrage de référence accessible sur les méthodes d'interprétabilité, dont SHAP, les importances de variables et les explications locales.

Applications actuarielles et assurance

- Wüthrich, M. V., & Merz, M. (2023). *Statistical Foundations of Actuarial Learning and its Applications*. Springer. Référence actuarielle moderne sur l'utilisation de méthodes statistiques et machine learning en assurance.
- Denuit, M., Hainaut, D., & Trufin, J. (2019). *Effective Statistical Learning Methods for Actuaries*. Springer. Ouvrage pertinent pour relier apprentissage statistique et problématiques actuarielles.
- Henckaerts, R., Côté, M. P., Antonio, K., & Verbelen, R. (2021). Boosting insights in insurance tariff plans with tree-based machine learning methods. *North American Actuarial Journal*, 25(2), 255-285. Article utile pour discuter des modèles d'arbres boostés en assurance et de leur interprétation.

Agents IA et grands modèles de langage

- Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*. Référence majeure sur les grands modèles de langage modernes.
- OpenAI. (2023). GPT-4 Technical Report. Rapport technique présentant les capacités et limites générales des modèles de langage avancés.
- Bommasani, R., et al. (2021). On the Opportunities and Risks of Foundation Models. *arXiv*. Référence utile pour discuter des opportunités, risques et enjeux de gouvernance des modèles de fondation.

Sommaire des annexes

Annexe A — Figures complémentaires essentielles

- Figure A.1 — Distribution de la cible
- Figure A.2 — Effet de SMOTE sur la balance des classes
- Figure A.3 — Matrice de confusion
- Figure A.4 — Courbe ROC
- Figure A.5 — Courbe précision/rappel
- Figure A.6 — F1-score selon le seuil
- Figure A.7 — Distribution bootstrap de la ROC AUC
- Figure A.8 — Comparaison des champions complet et sans variables tarifaires
- Figure A.9 — Importance globale SHAP

Annexe B — Tableaux de synthèse complémentaires

- Tableau B.1 — Effet de SMOTE sur la distribution des classes
- Tableau B.2 — Audit synthétique des données d'entraînement
- Tableau B.3 — Proportions selon l'intensité de SMOTE
- Tableau B.4 — Classement synthétique des configurations
- Tableau B.5 — Seuils calibrés par configuration
- Tableau B.6 — Classement synthétique des modèles sans variables tarifaires

Annexe C — Hyperparamètres et reproductibilité

- Meilleurs hyperparamètres par modèle

Annexe A — Figures complémentaires essentielles

Cette annexe conserve uniquement les figures nécessaires à la lecture du mémoire. Les autres sorties graphiques restent disponibles dans le dossier `reports/figures/`.

Figure A.1 — Distribution de la cible

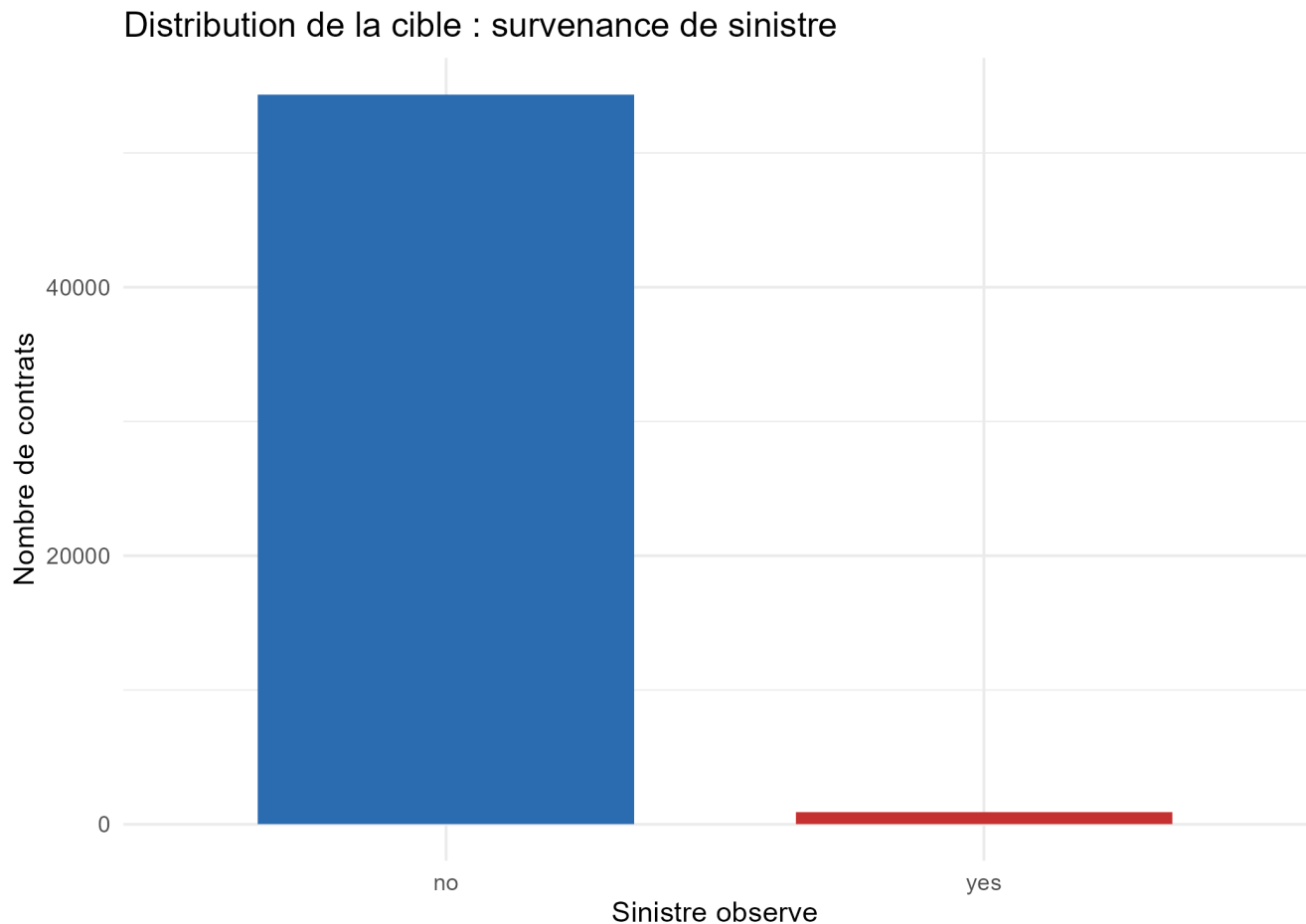


Figure A.2 — Effet de SMOTE sur la balance des classes

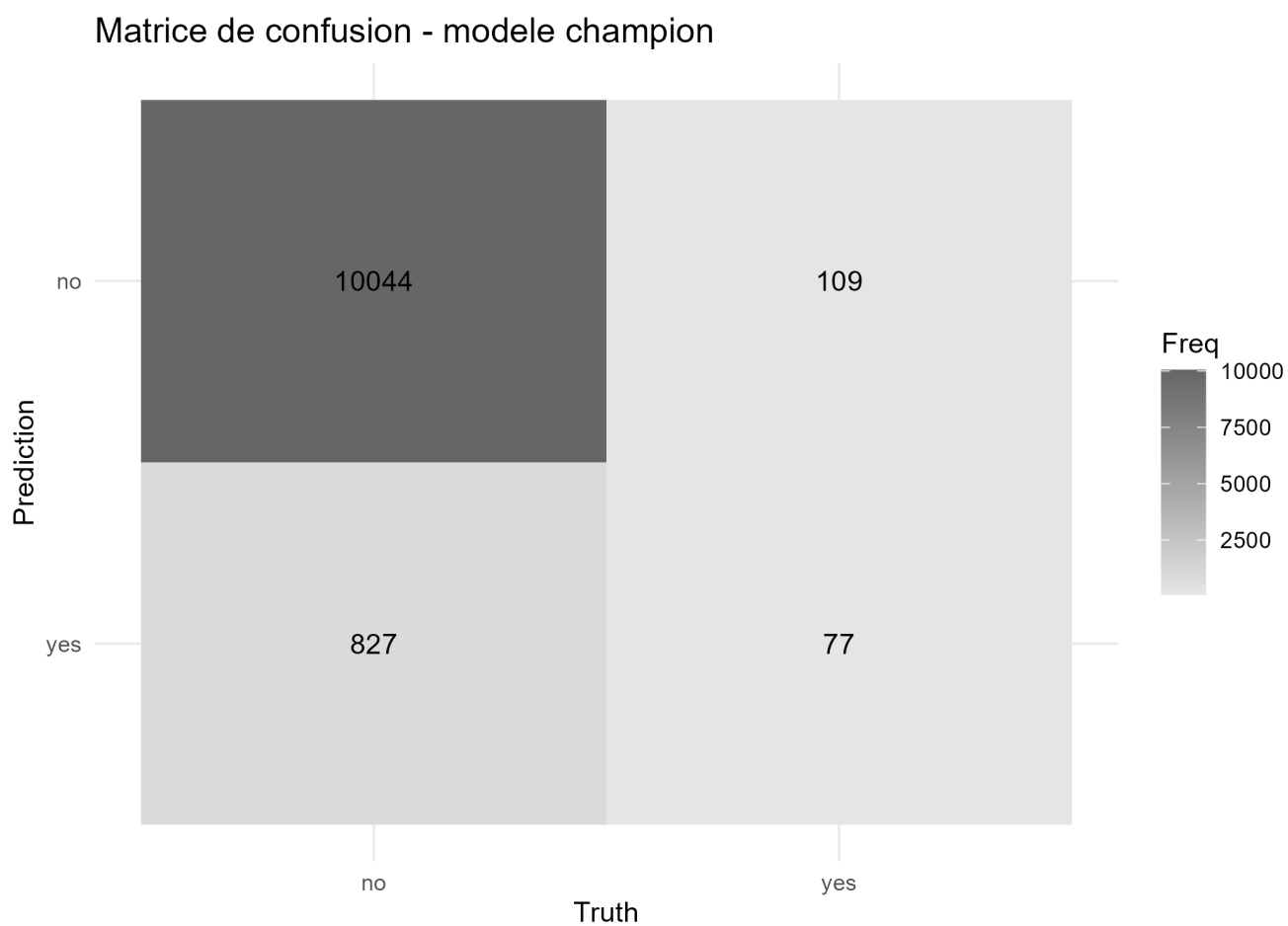
Figure A.3 — Matrice de confusion

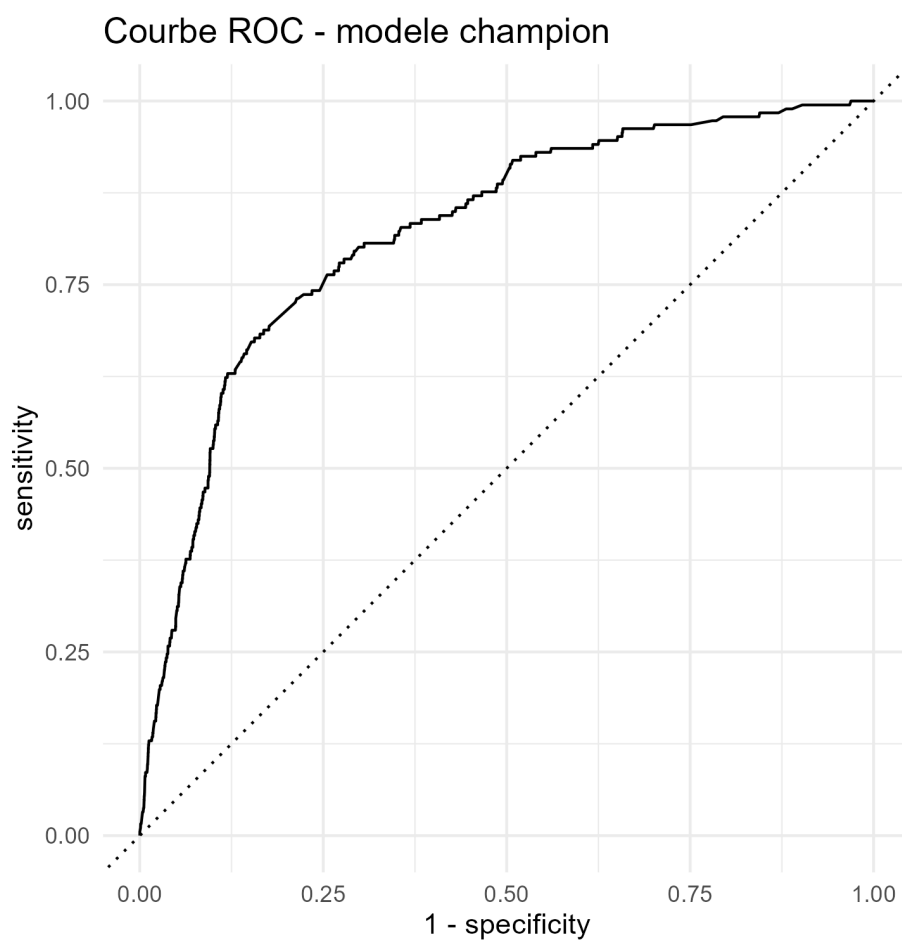
Figure A.4 — Courbe ROC

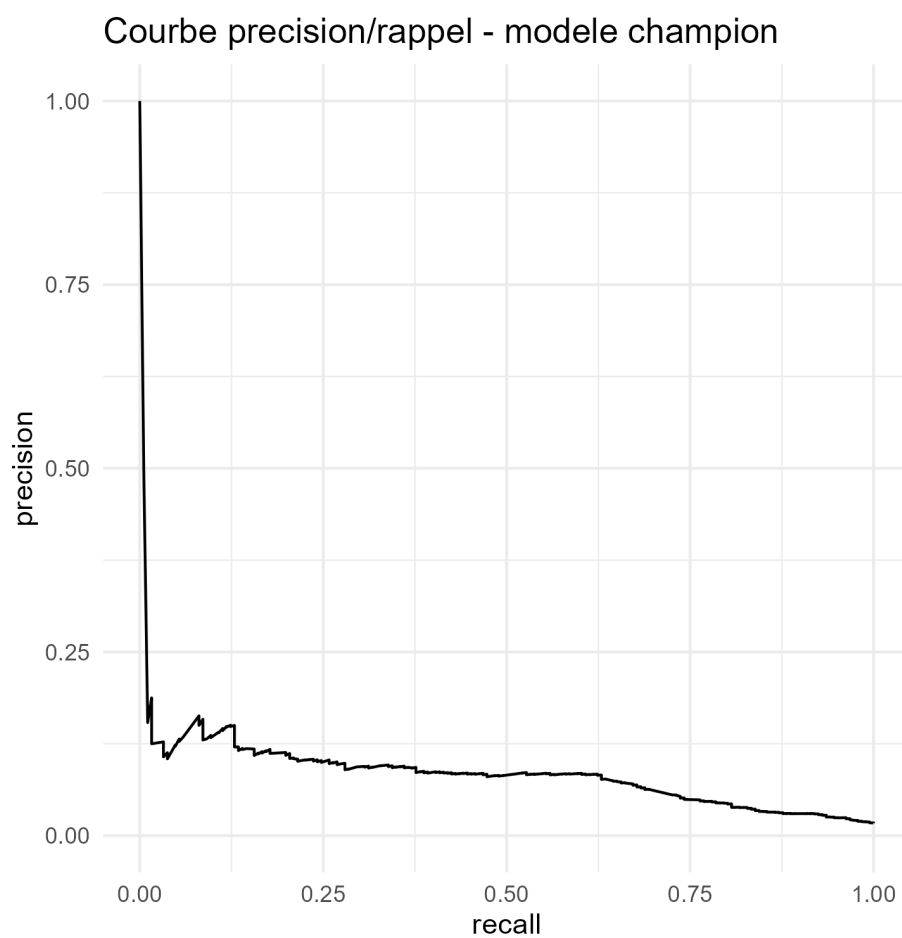
Figure A.5 — Courbe précision/rappel

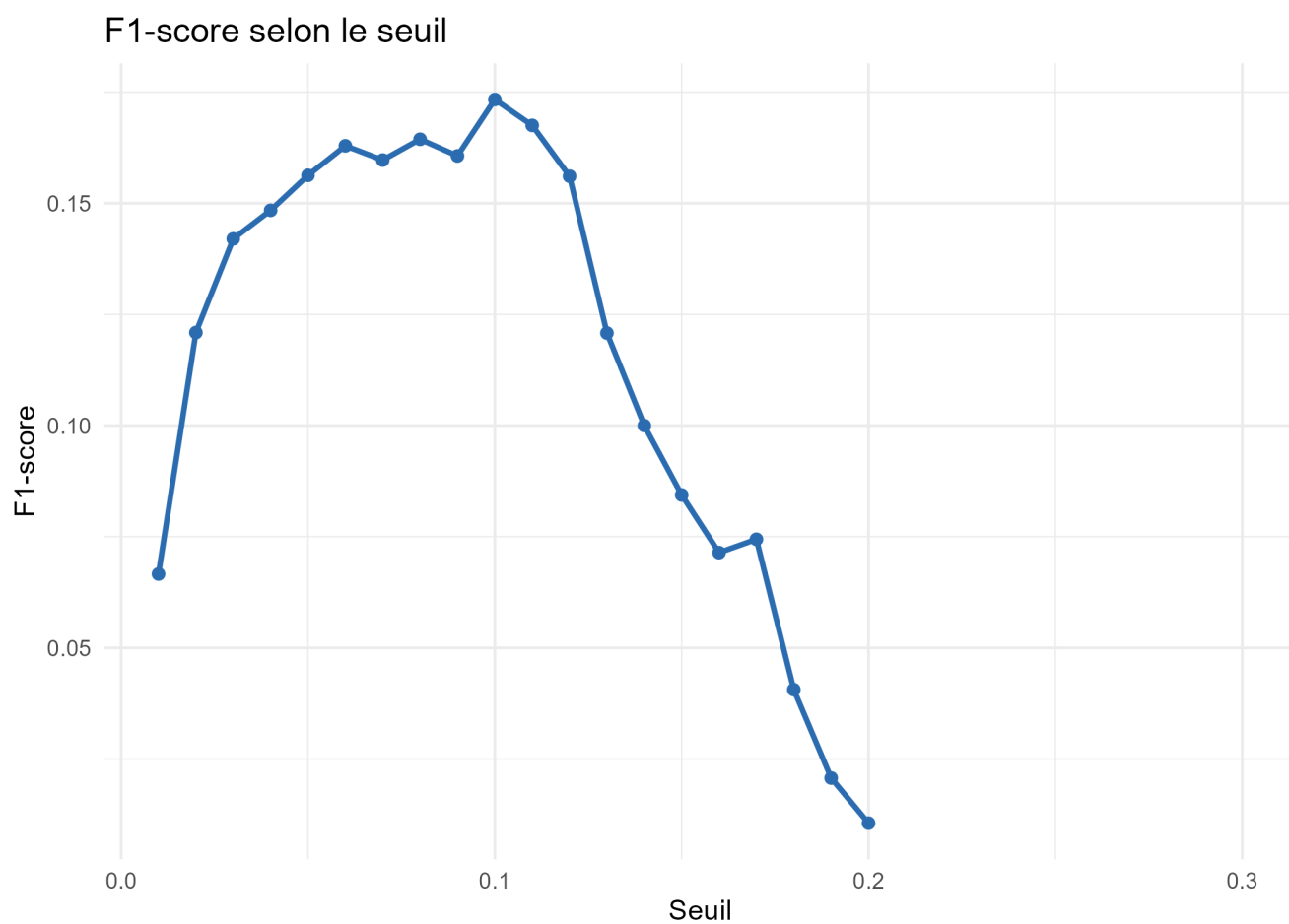
Figure A.6 — F1-score selon le seuil

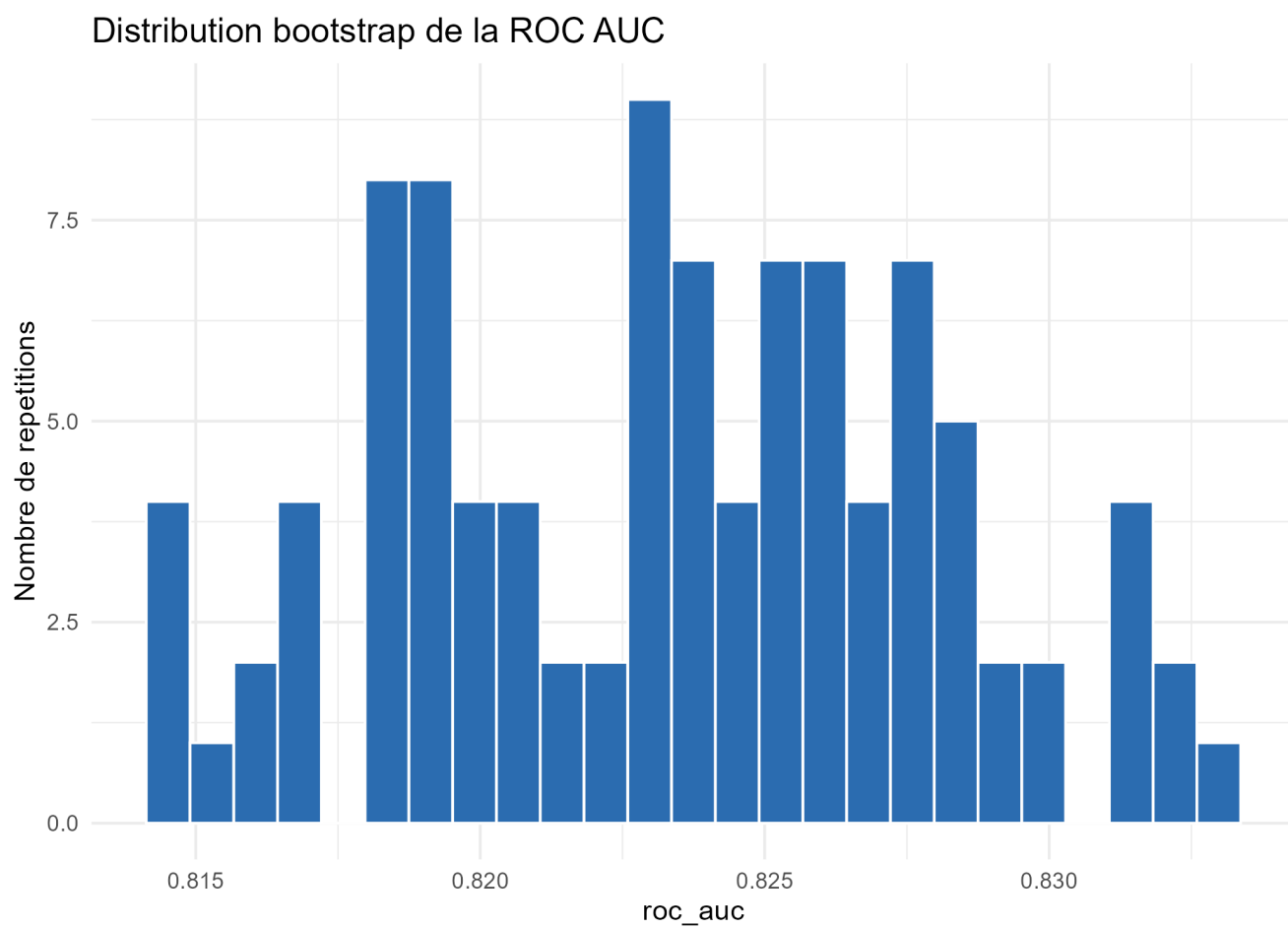
Figure A.7 — Distribution bootstrap de la ROC AUC

Figure A.8 — Comparaison des champions complet et sans variables tarifaires

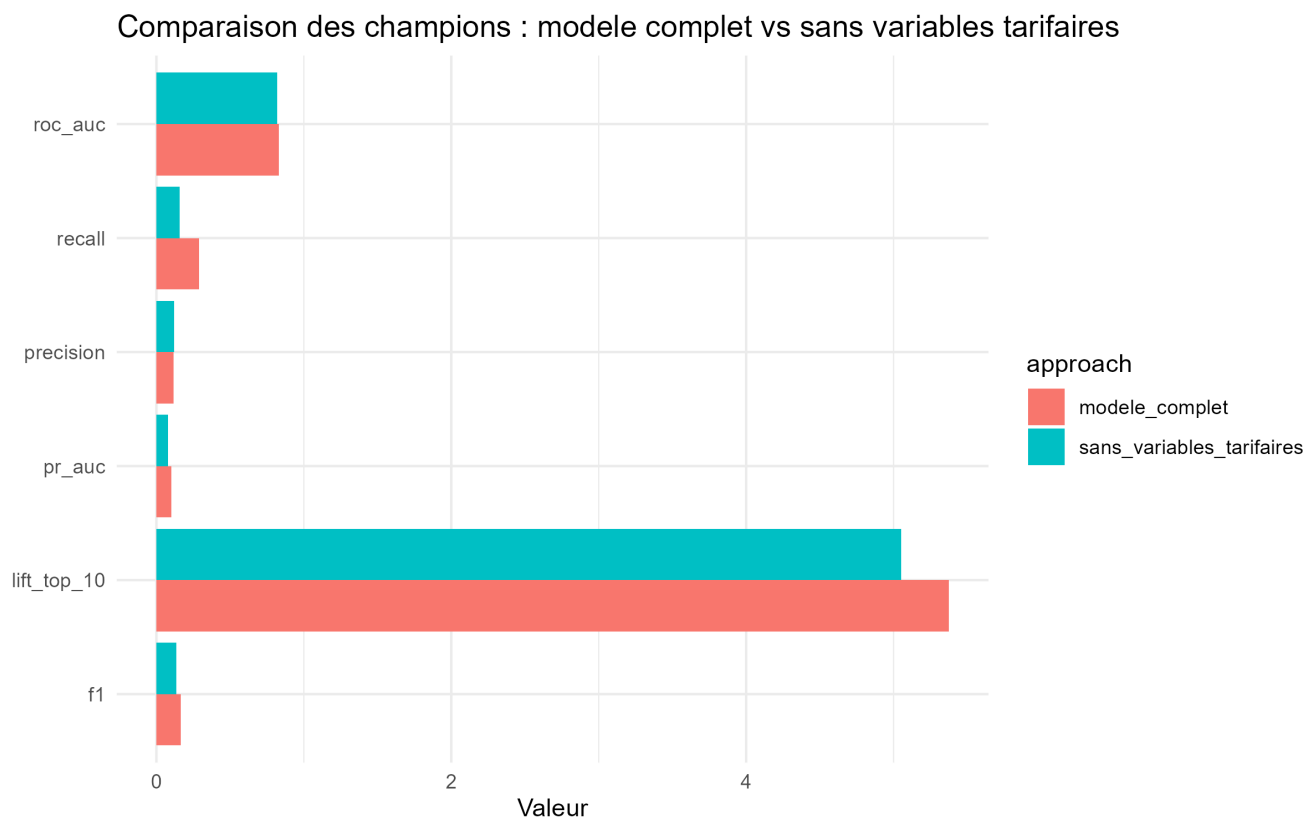
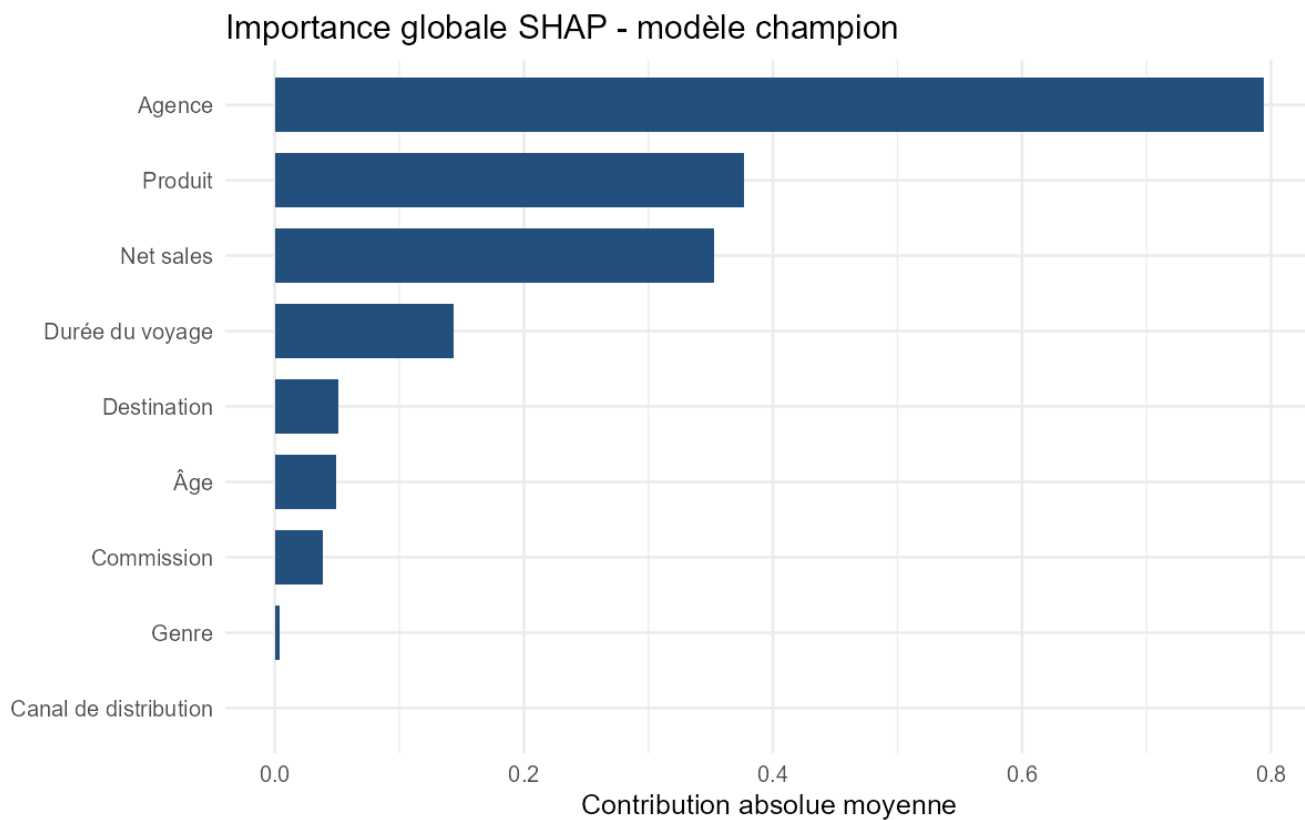


Figure A.9 — Importance globale SHAP

Annexe B — Tableaux de synthèse complémentaires

Les tableaux suivants conservent les contrôles méthodologiques utiles sans reprendre toutes les sorties techniques du projet.

Tableau B.1 — Effet de SMOTE sur la distribution des classes

Stage	Strategie	No	Yes	% yes	Total
full_data_original	none	54363	921	0.0167	55284
train_before_smote	none	43492	735	0.0166	44227
train_after_smote	smote	43492	13047	0.2308	56539
test_unchanged	none	10871	186	0.0168	11057

Tableau B.2 — Audit synthétique des données d'entraînement

Strategie	SMOTE	Lignes train	Yes train	% yes train	Lignes test	% yes test
none	Non	44227	735	0.0166	11057	0.0168
smote	Oui	56539	13047	0.2308	11057	0.0168

Tableau B.3 — Proportions selon l'intensité de SMOTE

Strategie	over_ratio	Yes train	% yes train	Total train	% yes test
none	NA	735	0.0166	44227	0.0168
smote_10	0.111	4827	0.0999	48319	0.0168
smote_20	0.250	10873	0.2000	54365	0.0168

Tableau B.4 — Classement synthétique des configurations

Modele	Strategie	F1	Recall	ROC AUC
xgboost	smote	0.1429	0.4435	0.8090
xgboost	none	0.1446	0.2054	0.8073
logistic_regression	none	0.1501	0.2626	0.8011
logistic_regression	smote	0.1375	0.4857	0.7959
random_forest	smote	0.1384	0.4544	0.7923
random_forest	none	0.1404	0.2259	0.7909
decision_tree	smote	0.1396	0.4762	0.7602
decision_tree	none	0.0327	1.0000	0.4892

Tableau B.5 — Seuils calibrés par configuration

Modele	Strategie	Seuil	Precision	Recall	F1
logistic_regression	none	0.07	0.1051	0.2626	0.1501
decision_tree	none	0.01	0.0166	1.0000	0.0327
random_forest	none	0.08	0.1018	0.2259	0.1404
xgboost	none	0.09	0.1116	0.2054	0.1446
logistic_regression	smote	0.50	0.0801	0.4857	0.1375
decision_tree	smote	0.43	0.0818	0.4762	0.1396
random_forest	smote	0.49	0.0816	0.4544	0.1384
xgboost	smote	0.48	0.0851	0.4435	0.1429

Tableau B.6 — Classement synthétique des modèles sans variables tarifaires

Modele	Strategie	F1	Recall	ROC AUC	PR AUC
xgboost	none	0.1362	0.1559	0.8179	0.0795
logistic_regression	none	0.1205	0.5968	0.8122	0.0710
catboost	weighted	0.1262	0.6452	0.8119	0.0844
random_forest	none	0.1377	0.1828	0.8118	0.0751
catboost	none	0.1562	0.1667	0.8068	0.0787
random_forest	smote	0.1364	0.2258	0.8056	0.0766
decision_tree	smote	0.1335	0.3280	0.7701	0.1600
xgboost	smote	0.0200	0.0215	0.6096	0.0237
decision_tree	none	0.0331	1.0000	0.5000	0.5084
logistic_regression	smote	0.0130	0.0968	0.4302	0.0595

Annexe C — Hyperparamètres et reproductibilité

Cette annexe conserve la synthèse des hyperparamètres retenus. Les résultats détaillés de tuning restent disponibles dans les fichiers CSV du dossier **reports/**.

Meilleurs hyperparamètres par modèle

Modele	Strategie	Metrique selection	Valeur
xgboost	xgboost_none	cv_roc_auc	0.8109
xgboost	xgboost_weighted	cv_roc_auc	0.8109
xgboost	xgboost_smote_light	cv_roc_auc	0.8100
random_forest	random_forest	cv_roc_auc	0.7697
catboost	none	cv_roc_auc	0.7910
catboost	smote	cv_roc_auc	0.8133
catboost	weighted	cv_roc_auc	0.8126